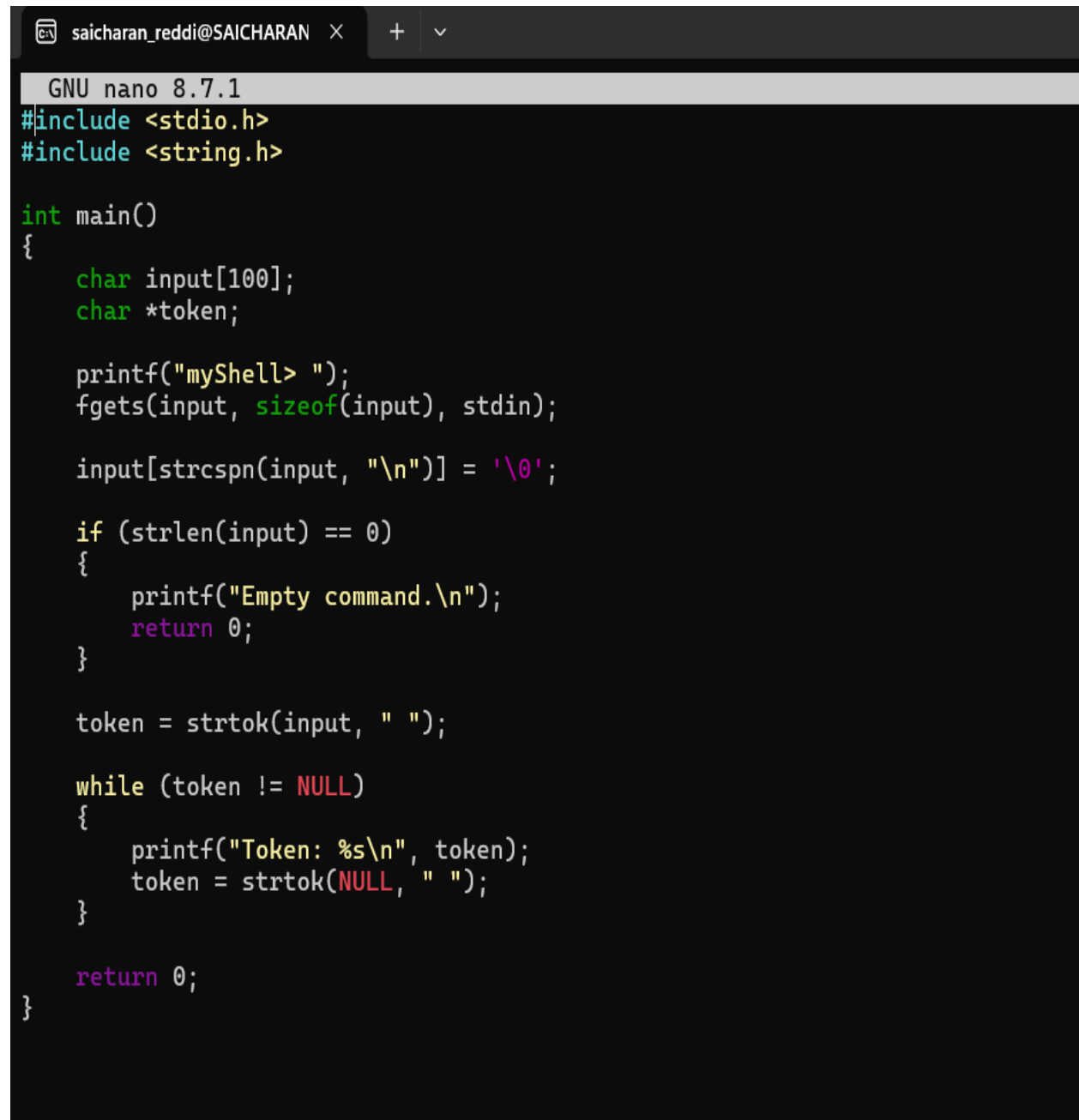


SKILL WEEK-4

Task 1:

To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output and Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures

CODE:

A screenshot of a terminal window with a dark background. The window title bar shows a browser tab with the address 'saicharan_reddi@SAICHARAN'. The terminal header indicates 'GNU nano 8.7.1'. The code is written in C and uses syntax highlighting: keywords like 'include', 'int', 'if', 'while', 'return', and 'NULL' are in green; string literals and character literals are in yellow; and variable names and function names are in white. The code implements a simple shell that reads a line of input, strips the trailing newline, and then tokenizes it by spaces. It prints each token on a new line. If the input is empty, it prints 'Empty command.\n' and returns 0. The code is enclosed in a 'main' function.

```
GNU nano 8.7.1
#include <stdio.h>
#include <string.h>

int main()
{
    char input[100];
    char *token;

    printf("myShell> ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0)
    {
        printf("Empty command.\n");
        return 0;
    }

    token = strtok(input, " ");

    while (token != NULL)
    {
        printf("Token: %s\n", token);
        token = strtok(NULL, " ");
    }

    return 0;
}
```

OUTPUT:

```
saicharan_reddi@SAICHARAN x + v
saicharan_reddi@SAICHARANREDDY:~$ nano Parsing.c
saicharan_reddi@SAICHARANREDDY:~$ gcc Parsing.c -o Parsing
saicharan_reddi@SAICHARANREDDY:~$ ./Parsing
myShell> |
```

REPORT: The input was successfully divided into individual tokens using delimiters such as spaces and tabs. The program correctly displayed each argument separately and handled empty commands. Thus, the basic tokenization and parsing functionality required for the shell was successfully implemented.

Task 2:

To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

```
GNU nano 8.7.1
#include <stdio.h>
#include <string.h>

int main()
{
    char input[200];
    int count = 0;

    printf("myShell> ");
    fgets(input, sizeof(input), stdin);

    for (int i = 0; input[i] != '\0'; i++)
    {
        if (input[i] == '\')
            count++;
    }

    if (count % 2 != 0)
        printf("Syntax Error: Unmatched single quote.\n");
    else
        printf("Parsing successful.\n");

    return 0;
}
```

OUTPUT:

```
saicharan_reddi@SAICHARANREDDY:~$ nano singlequotes.c
saicharan_reddi@SAICHARANREDDY:~$ gcc singlequotes.c -o singlequotes
saicharan_reddi@SAICHARANREDDY:~$ ./singlequotes
myShell> echo 'Hello World
Syntax Error: Unmatched single quote.
saicharan_reddi@SAICHARANREDDY:~$ |
```

REPORT: The Single-quoted strings were successfully detected and processed. The literal content inside the quotes was preserved, variable expansion was not performed, and unmatched single quotes were detected and reported as syntax errors.

Task 3:

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

CODE:

```
GNU nano 8.7.1
#include <stdio.h>
#include <string.h>

int main()
{
    char input[200];
    int count = 0;

    printf("myShell> ");
    fgets(input, sizeof(input), stdin);

    for (int i = 0; input[i] != '\0'; i++)
    {
        if (input[i] == '"')
            count++;
    }

    if (count % 2 != 0)
        printf("Syntax Error: Unmatched double quote.\n");
    else
        printf("Parsing successful.\n");

    return 0;
}|
```

OUTPUT:

```
saicharan_reddi@SAICHARANREDDY:~$ nano Doublequotes.c
saicharan_reddi@SAICHARANREDDY:~$ gcc Doublequotes.c -o Doublequotes
saicharan_reddi@SAICHARANREDDY:~$ ./Doublequotes
myShell> echo 'Hello World'
Parsing successful.
saicharan_reddi@SAICHARANREDDY:~$ |
```

REPORT: The Double-quoted input was successfully identified and parsed. Spaces inside quotes were preserved, variable expansion was supported conceptually, and unmatched double quotes were detected as syntax errors.

Task 4: To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

CODE:

```
GNU nano 8.7.1
#include <stdio.h>

int main()
{
    char input[200];
    char output[200];
    int i, j = 0;

    printf("myShell> ");
    fgets(input, sizeof(input), stdin);

    for (i = 0; input[i] != '\0'; i++)
    {
        if (input[i] == '\\' && input[i + 1] != '\0')
        {
            i++;
            output[j++] = input[i];
        }
        else
        {
            output[j++] = input[i];
        }
    }

    output[j] = '\0';

    printf("Parsed Input: %s", output);

    return 0;
}
```

OUTPUT:

```
saicharan_reddi@SAICHARANREDDY:~$ nano Escape.c
saicharan_reddi@SAICHARANREDDY:~$ gcc Escape.c -o Escape
saicharan_reddi@SAICHARANREDDY:~$ ./Escape
myShell> echo 'Hello World
Parsed Input: echo 'Hello World
saicharan_reddi@SAICHARANREDDY:~$ |
```

REPORT: Escape sequences were successfully handled. The program preserved escaped spaces and special characters and produced the corresponding parsed output.

Task 5: To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

CODE:

```
GNU nano 8.7.1 Execute.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    pid_t pid;
    int status;

    printf("mysHELL$ ls\n");

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child Process PID: %d\n", getpid());

        execlp("ls", "ls", NULL);

        perror("Execution failed");
        exit(1);
    }
    else
    {
        printf("Parent Process PID: %d\n", getpid());
        printf("Created Child PID: %d\n", pid);

        wait(&status);

        printf("Child exited with status: %d\n",
```

```
printf("Child exited with status: %d\n",
      WEXITSTATUS(status));
}

return 0;
}
```

OUTPUT:

```
saicharan_reddi@SAICHARANREDDY:~$ nano Execute.c
saicharan_reddi@SAICHARANREDDY:~$ gcc Execute.c -o Execute
saicharan_reddi@SAICHARANREDDY:~$ ./Execute
mysHELL$ ls
Parent Process PID: 828
Child Process PID: 829
Created Child PID: 829
BufferSize.c.save  Escape      ForkProgram.c  PROCESSTASK.save  SystemCall      command_executor  filecopy.c      process.c      snap
ChildProcess      Escape.c        LICENSE       ParentChild      SystemCall.c     command_executor.c  fliecopy.c.save  scripts      src
ChildProcess.c    Execute        Makefile      ParentChild.c    SystemCallParent  commandhistory.c   include          shell         tests
Command.c         Execute.c      MemoryList    Parsing.c         assets           copyfile.c         interactiveLoop  shell.c
DisplayPID        File1.txt     MemoryList.c  Parsing.c         bin              docs               interactiveLoop.c  shellcommand
DisplayPID.c      File2.txt     OSSP          ProcessParent    command          file               kill.c           shellcommand.c
DoubleQuotes      FileManagement  OSSP_old     ProcessParent.c  command         file              obj              singlequotes
DoubleQuotes.c    ForkProgram   OS_LAB       README.md        command.c       file.c            process          singlequotes.c
Child exited with status: 0
saicharan_reddi@SAICHARANREDDY:~$ |
```

REPORT: The child process was successfully created and used to execute a command. The parent process successfully waited for the child and obtained its exit status. Execution errors were also handled.